

Nombre de la práctica	Visualización del DataSet			No.	6
Asignatura:	Simulación	Carrera:	ISIC	Duración de la práctica (Hrs)	2

I. Competencia(s) específica(s): Aprender a realizar visualización del DataSet con python

II. Lugar de realización de la práctica (laboratorio, taller, aula u otro): Aula

III. Material empleado:

Computadora

Python

IV. Desarrollo de la práctica:

Visualización del DataSet

En este Notebook se muestran algunos de los mecanismos mas utilizados para la visualización de datos.

Abstract

NSL-KDD is a data set suggested to solve some of the inherent problems of the KDD'99 data set which are mentioned in [1]. Although, this new version of the KDD data set still suffers from some of the problems discussed by McHugh [2] and may not be a perfect representative of existing real networks, because of the lack of public data sets for network-based IDSs, we believe it still can be applied as an effective benchmark data set to help researchers compare different intrusion detection methods. Furthermore, the number of records in the NSL-KDD train and test sets are reasonable. This advantage makes it affordable to run the experiments on the complete set without the need to randomly select a small portion. Consequently, evaluation results of different research work will be consistent and comparable.

Data Files

- **KDDTrain+.ARFF** - The full NSL-KDD train set with binary labels in ARFF format
- **KDDTrain+.TXT** - The full NSL-KDD train set including attack-type labels and difficulty level in CSV format
- **KDDTrain+_20Percent.ARFF** - A 20% subset of the KDDTrain+.arff file
- **KDDTrain+_20Percent.TXT** - A 20% subset of the KDDTrain+.txt file
- **KDDTest+.ARFF** - The full NSL-KDD test set with binary labels in ARFF format
- **KDDTest+.TXT** - The full NSL-KDD test set including attack-type labels and difficulty level in CSV format
- **KDDTest-21.ARFF** - A subset of the KDDTest+.arff file which does not include records with difficulty level of 21 out of 21
- **KDDTest-21.TXT** - A subset of the KDDTest+.txt file which does not include records with difficulty level of 21 out of 21



```
import pandas as pd
df = pd.read_csv("datasets/NSL-KDD/KDDTrain+.txt")
df
```

	0	tcp	ftp_data	SF	491	0.1	0.2	0.3	0.4	0.5	...	0.17	0.03	0.17.1	0.00.6	0.00.7	0.00.8	0.05	0.00.9	normal	20	
0	0	udp	other	SF	146	0	0	0	0	0	...	0.00	0.60	0.88	0.00	0.00	0.00	0.00	0.00	0.00	normal	15
1	0	tcp	private	S0	0	0	0	0	0	0	...	0.10	0.05	0.00	0.00	1.00	1.00	0.00	0.00	0.00	neptune	19
2	0	tcp	http	SF	232	8153	0	0	0	0	...	1.00	0.00	0.03	0.04	0.03	0.01	0.00	0.01	0.00	normal	21
3	0	tcp	http	SF	199	420	0	0	0	0	...	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	normal	21
4	0	tcp	private	REJ	0	0	0	0	0	0	...	0.07	0.07	0.00	0.00	0.00	0.00	1.00	1.00	0.00	neptune	21
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
125967	0	tcp	private	S0	0	0	0	0	0	0	...	0.10	0.06	0.00	0.00	1.00	1.00	0.00	0.00	0.00	neptune	20
125968	8	udp	private	SF	105	145	0	0	0	0	...	0.96	0.01	0.01	0.00	0.00	0.00	0.00	0.00	0.00	normal	21
125969	0	tcp	smtp	SF	2231	384	0	0	0	0	...	0.12	0.06	0.00	0.00	0.72	0.00	0.01	0.00	0.00	normal	18
125970	0	tcp	klogin	S0	0	0	0	0	0	0	...	0.03	0.05	0.00	0.00	1.00	1.00	0.00	0.00	0.00	neptune	20
125971	0	tcp	ftp_data	SF	151	0	0	0	0	0	...	0.30	0.03	0.30	0.00	0.00	0.00	0.00	0.00	0.00	normal	21

125972 rows × 43 columns



```
df["attributes"]
```

```
'finger',  
'ftp',  
'ftp_data',  
'gopher',  
'harvest',  
'hostnames',  
'http',  
'http_2784',  
'http_443',  
'http_8001',  
'imap4',  
'IRC',  
'iso_tsap',  
'klogin',  
'kshell',  
'ldap',  
'link',  
'login',  
'...
```

```
# Parsear los atributos y obtener unicamente los nombres  
atributos = [attr[0] for attr in df['attributes']]  
atributos
```

```
['duration',  
'protocol_type',  
'service',  
'flag',  
'src_bytes',  
'dst_bytes',  
'land',  
'wrong_fragment',  
'urgent',  
'hot',  
'num_failed_logins',  
'logged_in',  
'num_compromised',  
'root_shell',  
'su_attempted',  
'num_root',  
'num_file_creations',  
'num_shells',  
'num_access_files',  
'num_outbound_cmds',  
'is_host_login',  
'is_guest_login',  
'count',  
'srv_count',  
'...
```

```
'error_rate',
'srv_error_rate',
'rerror_rate',
'srv_rerror_rate',
'same_srv_rate',
'diff_srv_rate',
'srv_diff_host_rate',
'dst_host_count',
'dst_host_srv_count',
'dst_host_same_srv_rate',
'dst_host_diff_srv_rate',
'dst_host_same_src_port_rate',
'dst_host_srv_diff_host_rate',
'dst_host_error_rate',
'dst_host_srv_error_rate',
'dst_host_rerror_rate',
'dst_host_srv_rerror_rate',
'class']
```

```
# Leer el DataSet con pandas y facilitar su manipulacion
df = pd.DataFrame(df['data'], columns = atributos)
df
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_srv_count	dst_host_same_srv_rate	dst_host_diff_srv_rate
0	0.0	tcp	ftp_data	SF	491.0	0.0	0	0.0	0.0	0.0	...	25.0	0.17	0.83
1	0.0	udp	other	SF	146.0	0.0	0	0.0	0.0	0.0	...	1.0	0.00	1.00
2	0.0	tcp	private	S0	0.0	0.0	0	0.0	0.0	0.0	...	26.0	0.10	0.90
3	0.0	tcp	http	SF	232.0	8153.0	0	0.0	0.0	0.0	...	255.0	1.00	0.00
4	0.0	tcp	http	SF	199.0	420.0	0	0.0	0.0	0.0	...	255.0	1.00	0.00
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
125968	0.0	tcp	private	S0	0.0	0.0	0	0.0	0.0	0.0	...	25.0	0.10	0.90
125969	8.0	udp	private	SF	105.0	145.0	0	0.0	0.0	0.0	...	244.0	0.96	0.04
125970	0.0	tcp	smtp	SF	2231.0	384.0	0	0.0	0.0	0.0	...	30.0	0.12	0.88
125971	0.0	tcp	klogin	S0	0.0	0.0	0	0.0	0.0	0.0	...	8.0	0.03	0.97
125972	0.0	tcp	ftp_data	SF	151.0	0.0	0	0.0	0.0	0.0	...	77.0	0.30	0.70

125973 rows × 42 columns

lo ideal es construir una funcion que permita leer el DataSet de manera mas limpia, este tipo de practicas son de gran utilidad para que nuestro codigo en Jupyter Notebook sea mas modular y pueda reutilizarse de manera mas sencilla para futuros despliegues.

```
def load_kdd_dataset(data_path):
    """Lectura del DataSet NLS-KDD"""
    with open(data_path, 'r') as train_set:
        dataset = arff.load(train_set)
        attributes = [attr[0] for attr in dataset['attributes']]
        return pd.DataFrame(dataset['data'], columns = attributes)
```

```
load_kdd_dataset('datasets/NSL-KDD/KDDTrain+.arff')
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_srv_count	dst_host_same_srv_rate	dst_host_di
0	0.0	tcp	ftp_data	SF	491.0	0.0	0	0.0	0.0	0.0	...	25.0	0.17	
1	0.0	udp	other	SF	146.0	0.0	0	0.0	0.0	0.0	...	1.0	0.00	
2	0.0	tcp	private	S0	0.0	0.0	0	0.0	0.0	0.0	...	26.0	0.10	
3	0.0	tcp	http	SF	232.0	8153.0	0	0.0	0.0	0.0	...	255.0	1.00	
4	0.0	tcp	http	SF	199.0	420.0	0	0.0	0.0	0.0	...	255.0	1.00	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
125968	0.0	tcp	private	S0	0.0	0.0	0	0.0	0.0	0.0	...	25.0	0.10	
125969	8.0	udp	private	SF	105.0	145.0	0	0.0	0.0	0.0	...	244.0	0.96	
125970	0.0	tcp	smtp	SF	2231.0	384.0	0	0.0	0.0	0.0	...	30.0	0.12	
125971	0.0	tcp	klogin	S0	0.0	0.0	0	0.0	0.0	0.0	...	8.0	0.03	
125972	0.0	tcp	ftp_data	SF	151.0	0.0	0	0.0	0.0	0.0	...	77.0	0.30	

125973 rows × 42 columns

2.- Funciones basicas de visulaizacion de datos.

- El proceso de visualizacion siempre debe realizarse sobre el training_set. Esto evita que nuestro entrenamiento genere intuiciones del test_set que se incorpore a nuestro modelo.
- Una buena practica es crear una copia de training_set y juagr con ella. De esta manera si se resalizan transformaciones que danen el training_set el original no se vera afectado

```
#lectura y copia del dataSet
df_orig = load_kdd_dataset('datasets/NSL-KDD/KDDTrain+.arff')
df = df_orig.copy()
```

```
df.head(10)
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_srv_count	dst_host_same_srv_rate	
0	0.0	tcp	ftp_data	SF	491.0	0.0	0	0.0	0.0	0.0	...	25.0	0.17	
1	0.0	udp	other	SF	146.0	0.0	0	0.0	0.0	0.0	...	1.0	0.00	
2	0.0	tcp	private	S0	0.0	0.0	0	0.0	0.0	0.0	...	26.0	0.10	
3	0.0	tcp	http	SF	232.0	8153.0	0	0.0	0.0	0.0	...	255.0	1.00	
4	0.0	tcp	http	SF	199.0	420.0	0	0.0	0.0	0.0	...	255.0	1.00	
5	0.0	tcp	private	REJ	0.0	0.0	0	0.0	0.0	0.0	...	19.0	0.07	
6	0.0	tcp	private	S0	0.0	0.0	0	0.0	0.0	0.0	...	9.0	0.04	
7	0.0	tcp	private	S0	0.0	0.0	0	0.0	0.0	0.0	...	15.0	0.06	
8	0.0	tcp	remote_job	S0	0.0	0.0	0	0.0	0.0	0.0	...	23.0	0.09	
9	0.0	tcp	private	S0	0.0	0.0	0	0.0	0.0	0.0	...	13.0	0.05	

10 rows × 42 columns



```
# Mostrar la informacion basica sobre el DataSet
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 125973 entries, 0 to 125972
Data columns (total 42 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   duration                             125973 non-null float64
1   protocol_type                        125973 non-null object
2   service                              125973 non-null object
3   flag                                 125973 non-null object
4   src_bytes                           125973 non-null float64
5   dst_bytes                           125973 non-null float64
6   land                                125973 non-null object
7   wrong_fragment                      125973 non-null float64
8   urgent                             125973 non-null float64
9   hot                                 125973 non-null float64
10  num_failed_logins                   125973 non-null float64
11  logged_in                           125973 non-null object
12  num_compromised                     125973 non-null float64
13  root_shell                          125973 non-null float64
14  su_attempted                       125973 non-null float64
15  num_root                            125973 non-null float64
16  num_file_creations                  125973 non-null float64
17  num_shells                          125973 non-null float64
```

```
#Mostrar las funciones estadisticas sobre el DataSet
df.describe()
```

	duration	src_bytes	dst_bytes	wrong_fragment	urgent	hot	num_failed_logins	num_compromised	root_shell
count	125973.00000	1.259730e+05	1.259730e+05	125973.000000	125973.000000	125973.000000	125973.000000	125973.000000	125973.000000
mean	287.14465	4.556674e+04	1.977911e+04	0.022687	0.000111	0.204409	0.001222	0.279250	0.001342
std	2604.51531	5.870331e+06	4.021269e+06	0.253530	0.014366	2.149968	0.045239	23.942042	0.036603
min	0.00000	0.000000e+00	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.00000	0.000000e+00	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
50%	0.00000	4.400000e+01	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
75%	0.00000	2.760000e+02	5.160000e+02	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
max	42908.00000	1.379964e+09	1.309937e+09	3.000000	3.000000	77.000000	5.000000	7479.000000	1.000000

8 rows x 34 columns

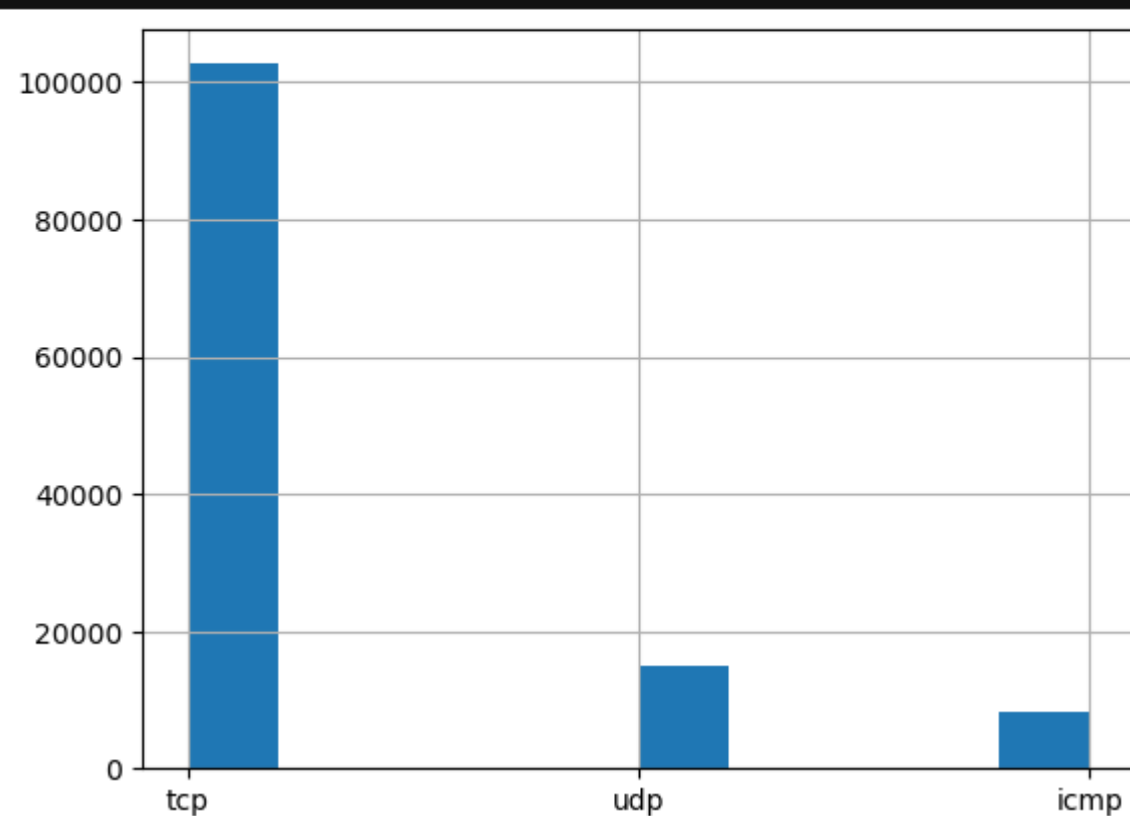


```
# Mostrar los valores unicos que tiene un atributo determinado  
df['protocol_type'].value_counts()
```

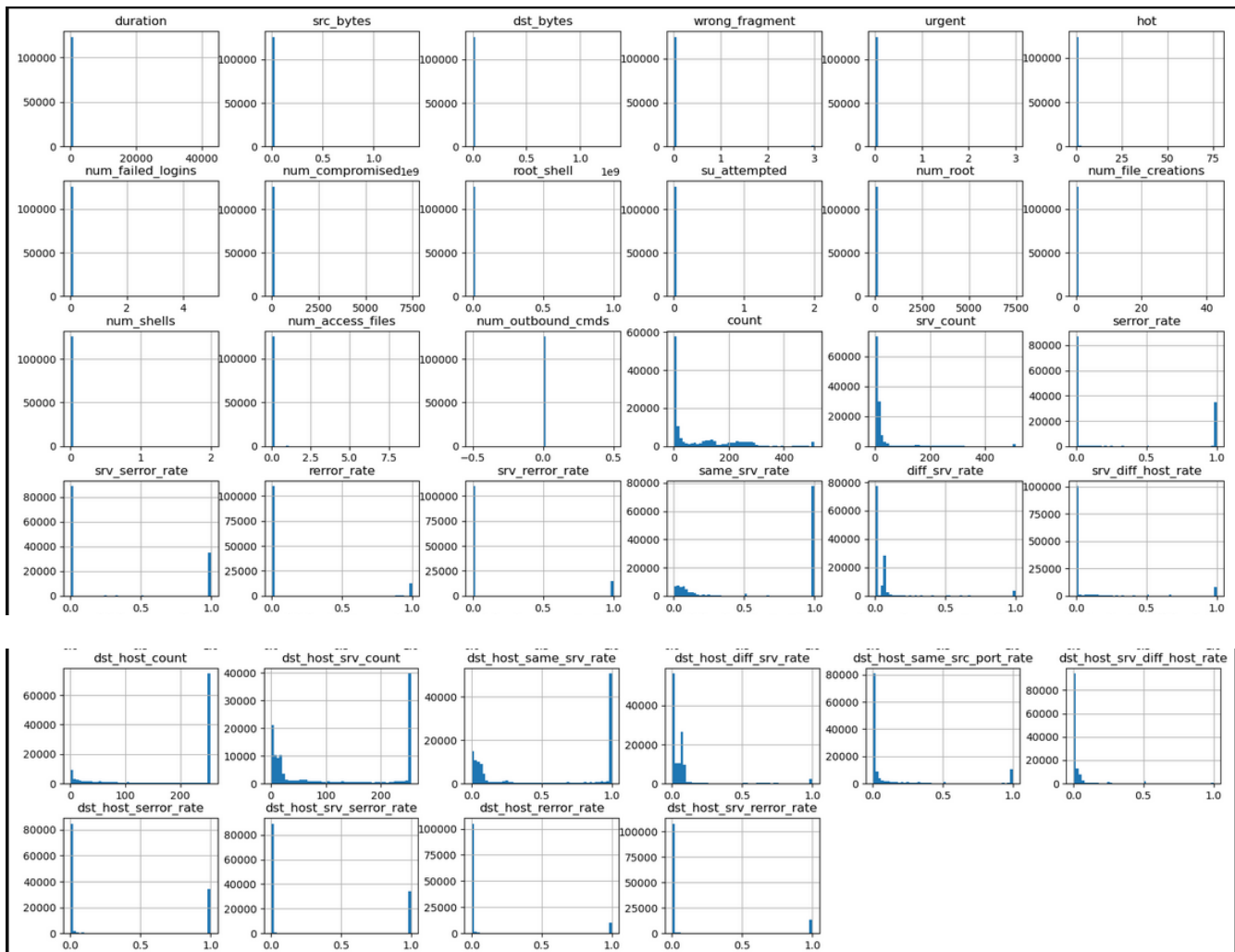
```
protocol_type  
tcp      102689  
udp       14993  
icmp       8291  
Name: count, dtype: int64
```

```
# Mostrar los valores de las características como un histograma  
%matplotlib inline  
import matplotlib.pyplot as plt  
df['protocol_type'].hist()
```

<Axes: >



```
# Representar graficamente la distribucion de los atributos.  
df.hist(bins = 50, figsize = (20,15))  
plt.show()
```



3.- Funciones Avanzadas de Visualización de los Datos

Buscando correlaciones

- Se puede calcular el coeficiente de correlación para evaluar la correlación entre cada parte de atributos.
- El coeficiente de correlación, solo mide **Correlaciones lineales**, esto quiere decir que si x va hacia arriba, medirá si y va hacia abajo o hacia
- Hay que intentar buscar correlaciones sobre todo con el atributo objetivo(el que se desea predecir), en este caso es class.



```
# El atributo class de nuestro DataSet tiene valores categoricos
df['class']
```

```
0      normal
1      normal
2      anomaly
3      normal
4      normal
```

```
...
125968  anomaly
125969  normal
125970  normal
125971  anomaly
125972  normal
```

```
Name: class, Length: 125973, dtype: object
```

```
# Trasformar los valores del atributo class de categoricos a numericos
```

```
from sklearn.preprocessing import LabelEncoder
labelencoder = LabelEncoder()
df['class'] = labelencoder.fit_transform(df['class'])
df
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_srv_count	dst_host_same_srv_rate
0	0.0	tcp	ftp_data	SF	491.0	0.0	0	0.0	0.0	0.0	...	25.0	0.17
1	0.0	udp	other	SF	146.0	0.0	0	0.0	0.0	0.0	...	1.0	0.00
2	0.0	tcp	private	S0	0.0	0.0	0	0.0	0.0	0.0	...	26.0	0.10
3	0.0	tcp	http	SF	232.0	8153.0	0	0.0	0.0	0.0	...	255.0	1.00
4	0.0	tcp	http	SF	199.0	420.0	0	0.0	0.0	0.0	...	255.0	1.00
...	...	...	...	...	...	...	...	...	...	...	...	...	...
125968	0.0	tcp	private	S0	0.0	0.0	0	0.0	0.0	0.0	...	25.0	0.10
125969	8.0	udp	private	SF	105.0	145.0	0	0.0	0.0	0.0	...	244.0	0.96
125970	0.0	tcp	smtp	SF	2231.0	384.0	0	0.0	0.0	0.0	...	30.0	0.12
125971	0.0	tcp	klogin	S0	0.0	0.0	0	0.0	0.0	0.0	...	8.0	0.03
125972	0.0	tcp	ftp_data	SF	151.0	0.0	0	0.0	0.0	0.0	...	77.0	0.30

125973 rows x 42 columns



```
# Mostrar la correlacion entre los atributos del DataSet  
corr_matrix = df.corr(numeric_only = True)  
corr_matrix['class'].sort_values(ascending = False)
```

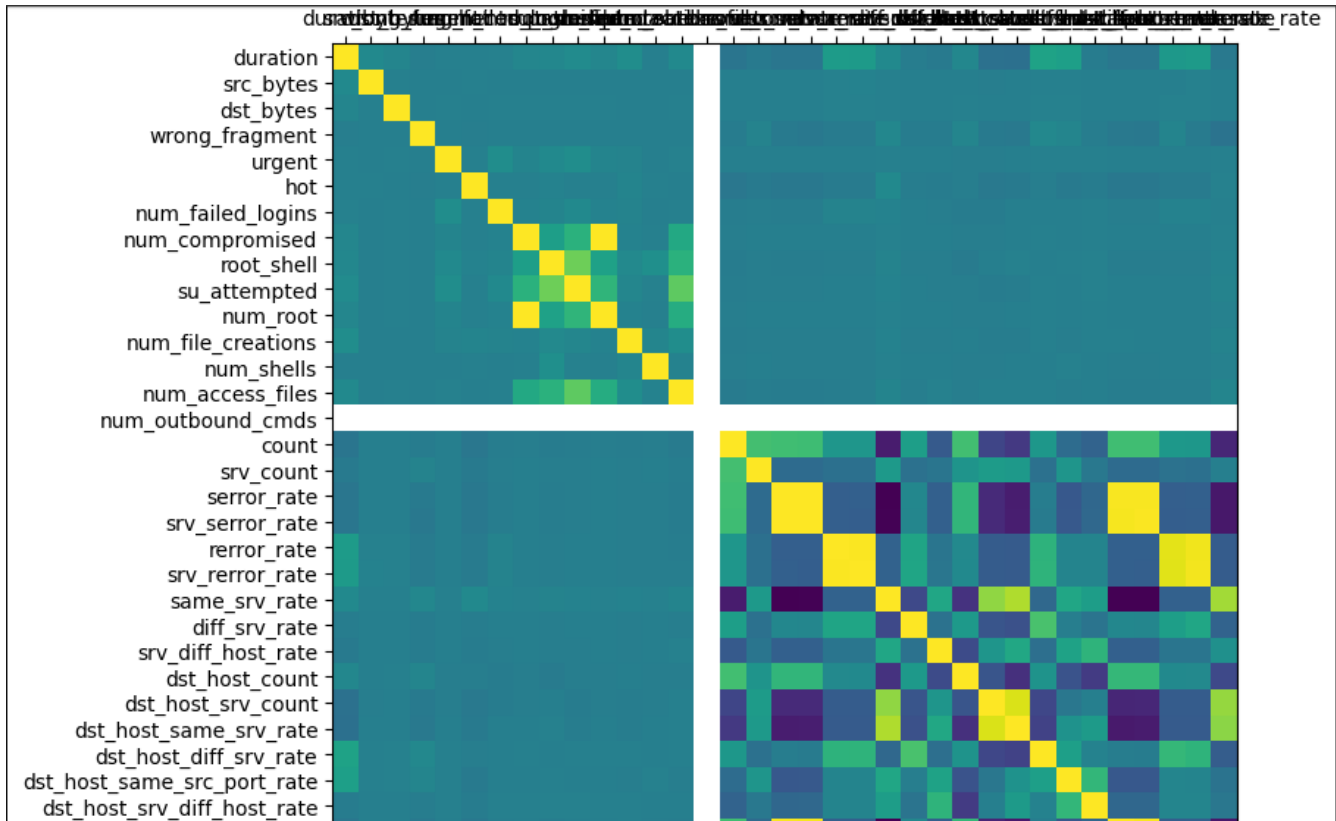
```
class                1.000000  
same_srv_rate        0.751913  
dst_host_srv_count    0.722535  
dst_host_same_srv_rate 0.693803  
srv_diff_host_rate    0.119377  
num_access_files      0.036701  
su_attempted          0.022448  
num_file_creations    0.021271  
root_shell            0.020285  
hot                   0.013083  
num_root              0.011452  
num_compromised        0.010198  
num_shells             0.009472  
num_failed_logins      0.003755  
urgent                0.002787  
srv_count              -0.000771  
dst_bytes              -0.004118  
src_bytes              -0.005921  
duration               -0.048785  
dst_host_srv_diff_host_rate -0.062332  
dst_host_same_src_port_rate -0.092444  
wrong_fragment         -0.095905  
diff_srv_rate          -0.203660  
dst_host_diff_srv_rate -0.242898  
dst_host_error_rate     -0.252563  
error_rate             -0.253397
```



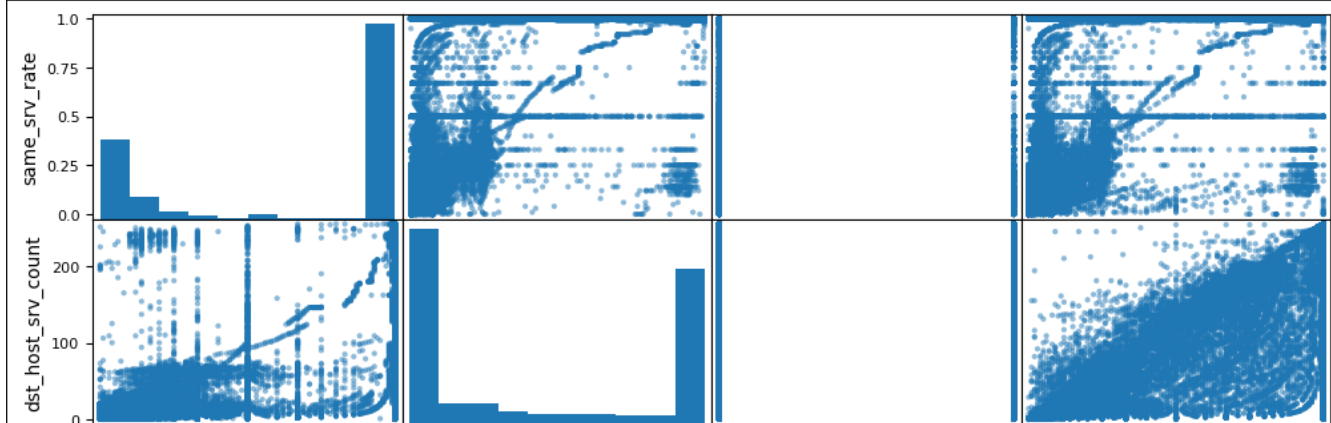
```
# Mostrar la correlacion lineal entre todos los atributos del DataSet
df.corr(numeric_only = True)
```

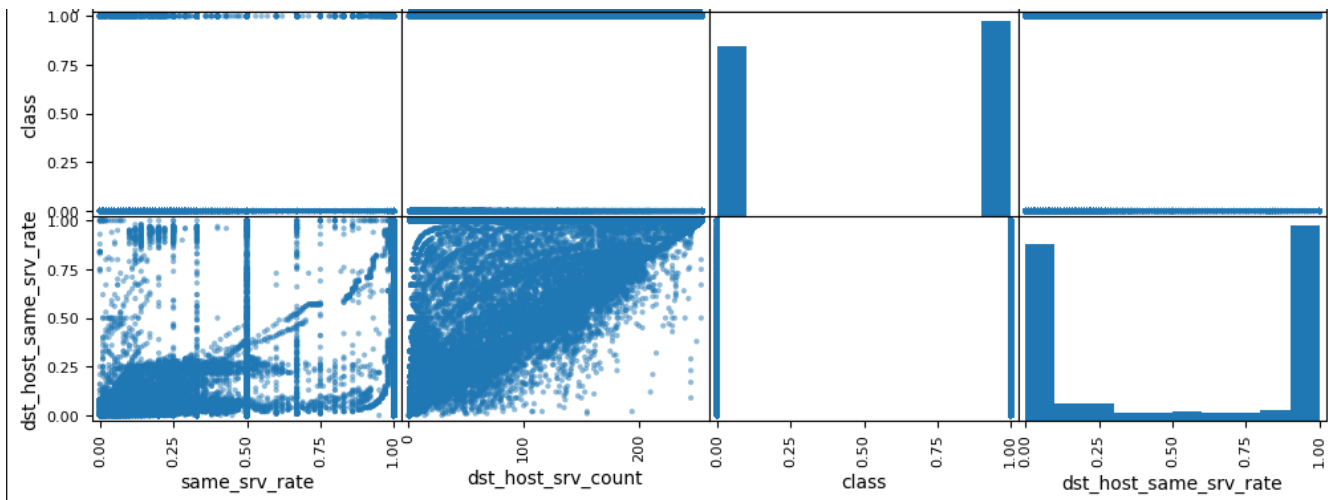
	duration	src_bytes	dst_bytes	wrong_fragment	urgent	hot	num_failed_logins	num_compromised	root_shell	su_attempted	...
duration	1.000000	0.070737	0.034878	-0.009866	0.003830	0.000705	0.009528	0.042679	0.052791	0.087183	...
src_bytes	0.070737	1.000000	0.000204	-0.000693	-0.000059	0.000295	-0.000208	-0.000086	-0.000272	-0.000186	...
dst_bytes	0.034878	0.000204	1.000000	-0.000440	0.000248	-0.000344	0.000504	0.001233	0.001069	0.001133	...
wrong_fragment	-0.009866	-0.000693	-0.000440	1.000000	-0.000692	-0.008508	-0.002418	-0.001044	-0.003280	-0.002187	...
urgent	0.003830	-0.000059	0.000248	-0.000692	1.000000	0.000293	0.097507	0.033329	0.075199	0.097710	...
hot	0.000705	0.000295	-0.000344	-0.008508	0.000293	1.000000	0.003715	0.002014	0.015379	0.000130	...
num_failed_logins	0.009528	-0.000208	0.000504	-0.002418	0.097507	0.003715	1.000000	0.019085	0.032567	0.073175	...
num_compromised	0.042679	-0.000086	0.001233	-0.001044	0.033329	0.002014	0.019085	1.000000	0.224872	0.362702	...
root_shell	0.052791	-0.000272	0.001069	-0.003280	0.075199	0.015379	0.032567	0.224872	1.000000	0.609083	...
su_attempted	0.087183	-0.000186	0.001133	-0.002187	0.097710	0.000130	0.073175	0.362702	0.609083	1.000000	...
num_root	0.045519	-0.000093	0.001229	-0.001108	0.032470	0.001510	0.018112	0.998833	0.243349	0.391038	...
num_file_creations	0.099116	-0.000179	0.000089	-0.002343	0.024918	0.028716	0.021774	0.015976	0.064919	0.044407	...
num_shells	-0.001593	-0.000134	-0.000083	-0.001665	-0.000144	0.004723	-0.000503	0.001338	0.116648	0.007471	...
num_access_files	0.070420	-0.000309	0.000339	-0.003689	0.010803	-0.001987	0.000652	0.299631	0.365152	0.565131	...
num_outbound_cmds	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...

```
# Representar Garficamente la matriz de correlacion
corr = df.corr(numeric_only = True)
fig, ax = plt.subplots(figsize = (8,8))
ax.matshow(corr)
plt.xticks(range(len(corr.columns)), corr.columns);
plt.yticks(range(len(corr.columns)), corr.columns);
```



```
# Representar graficamente las correlaciones
from pandas.plotting import scatter_matrix
attributes = ['same_srv_rate', 'dst_host_srv_count', 'class', 'dst_host_same_srv_rate']
scatter_matrix(df[attributes], figsize = (12, 8))
plt.show()
```





Conclusión:

La visualización del dataset en Python es clave para entender mejor cómo se comporta el modelo de predicción de correos spam y ham. Usando bibliotecas como matplotlib se pueden crear gráficos que muestran cómo se distribuyen las predicciones. Por ejemplo, se pueden graficar los correos en función de características clave como la frecuencia de palabras o la longitud del mensaje y colorearlos según la predicción del modelo. Esto permite ver claramente cómo el modelo separa los correos spam de los ham después de entrenarlo. Además, se pueden usar gráficos de barras o diagramas de dispersión para comparar los resultados reales con las predicciones del modelo y evaluar su precisión visualmente. Esto hace que sea más fácil ajustar parámetros o identificar patrones en los datos que puedan ayudar a mejorar la precisión del modelo.

NOMBRE DEL ALUMNO: Shania Kinnereth Diaz Moya

Grupo: 3501

Periodo: 2024-2

RÚBRICA DE EVALUACIÓN ISIC		Puntaje		 Comentarios
Tecnológico de Estudios Superiores de Jilotepec				
DOCUMENTO: Prácticas		Máxima	Obtenida	



Análisis de requerimientos Identifica los datos de relevancia y la problemática asociada.	20		
Diseño / modelado Representación gráfica o escrita de la solución	20		
Codificación / Configuración Traslado del modelado a un lenguaje de programación y/o configuración de la solución.	20		
Pruebas Comparar resultados obtenidos con los esperados	30		
Documentación de la práctica Entrega de manual de prácticas FO-ACA-11	10		
Tiempo de entrega No se califican entregables fuera de tiempo			
Calificación	100		